

THEORETICAL ASSIGNMENT 13

Apache Kafka • ZooKeeper • NiFi • PySpark Structured Streaming

Part 1 — Corrected Assignment Questions

1. What is Apache Kafka, and why is it needed when Delta Lake is already being used?
2. What is the difference between batch processing and stream processing?
3. What is the Publish–Subscribe (Pub/Sub) model? Explain it with an example.
4. Explain the roles of Apache Kafka, ZooKeeper, Producer, Consumer, Topic, and Broker. How do they work together?
5. What are Offset Explorer, PyCharm, and Apache NiFi? Explain the purpose of each tool.
6. Explain the steps to set up and run Kafka, ZooKeeper, PyCharm, Offset Explorer, and NiFi on a Windows laptop.
7. What is the startingOffsets option in PySpark Structured Streaming? Explain the difference between earliest and latest with an example.
8. What is checkpointLocation in PySpark Structured Streaming? Why is it required, and what information does it store?

1. What is Kafka, and why use it with Delta Lake?

Apache Kafka is a distributed event-streaming platform used to collect, store, and transmit data streams between applications in real time.

Kafka and Delta Lake serve different purposes. Kafka receives and distributes incoming events; Delta Lake stores data in tables and provides table-management features such as ACID transactions, schema enforcement, and history.

Example flow: An application publishes payment events to Kafka. PySpark reads and processes them, then writes the processed data to a Delta Lake table.

Application → Producer → Kafka Topic → PySpark Streaming → Delta Lake Table

Key point: Kafka handles event streaming and distribution; Delta Lake is a storage/table layer. They can be used together.

2. Batch Processing vs Stream Processing

Batch processing

Processes a collected group of records together, often at scheduled intervals. Example: processing all payments from the previous day.

Stream processing

Processes continuously arriving data, often with low latency. Example: processing payment events as they arrive.

Batch processing	Stream processing
Processes accumulated records	Processes continuously arriving data
Often scheduled	Runs continuously or in micro-batches
Daily reports, historical analysis	Live monitoring, timely analytics
Example: yesterday's transactions	Example: incoming transactions

3. Publish–Subscribe (Pub/Sub) Model

In Pub/Sub, publishers send messages to a topic and subscribers receive messages from that topic. The publisher and subscriber do not need to communicate directly.

Publisher → Kafka Topic → Consumer 1
→ Consumer 2

Example: an order event is published once to a topic and can be consumed by an analytics application and a notification application.

Key point: The publisher sends messages to the topic, not directly to each consumer.

4. Kafka Components

Apache Kafka

The platform that receives, stores, and distributes event data. In the class setup, the local broker uses port 9092.

ZooKeeper

A coordination service used by older Kafka deployments. In the class setup, it runs on port 2181. Newer Kafka deployments can use KRaft mode without ZooKeeper; these class commands are for the older ZooKeeper-based setup.

Producer

Sends messages to a Kafka topic. The class console producer sends each entered line to topic1.

Consumer

Reads messages from a topic according to its starting position and configuration.

Topic

A named stream of messages, such as topic1. A topic is divided into partitions; each partition contains an ordered sequence of messages, each with an offset.

Broker

A Kafka server that stores topic partitions and serves producer and consumer requests. The class uses one local broker at localhost:9092.

Component	Responsibility
Producer	Sends messages
Topic	Organizes messages by name
Broker	Stores and serves messages
Consumer	Reads messages
ZooKeeper	Coordinates older Kafka deployments

5. Offset Explorer, PyCharm, and NiFi

Offset Explorer

A graphical tool for inspecting Kafka clusters. It can display topics, partitions, messages, offsets, and consumer-group information, depending on the setup.

PyCharm

An IDE for writing, running, and debugging Python code. In this assignment, it can be used to run the PySpark Structured Streaming script.

Apache NiFi

A dataflow automation tool for designing and managing data movement between systems through a graphical interface. NiFi flows can connect to Kafka and other systems.

6. Windows Setup Steps

A. ZooKeeper

1. Download and extract ZooKeeper.
2. Open its conf folder and rename zoo_sample.cfg to zoo.cfg.
3. Open Command Prompt in the bin folder and run the startup script used by your class: .\zkserver
4. If that command is unavailable, inspect the distribution; standard Windows packages often use zkServer.cmd.

5. Keep the ZooKeeper terminal running.

B. Kafka broker

Open Command Prompt in the extracted Kafka folder and run:

```
.\bin\windows\kafka-server-start.bat .\config\server.properties
```

Keep the Kafka terminal running.

C. Create topic1

```
kafka-topics.bat --create --zookeeper localhost:2181 --replication-factor 1 --partitions 1 --topic topic1
```

This class command creates topic1 with one partition and replication factor 1. It uses older ZooKeeper-based administration syntax; newer Kafka versions generally use --bootstrap-server localhost:9092.

D. Start the producer

```
kafka-console-producer.bat --broker-list localhost:9092 --topic topic1
```

Type a message and press Enter to send it.

E. Start the consumer

```
kafka-console-consumer.bat --zookeeper localhost:2181 --topic topic1
```

The messages shown depend on the consumer's starting position and configuration.

F. PyCharm

Install PyCharm, configure a compatible Python environment, configure PySpark and its Kafka connector, then run the streaming script while Kafka is running.

G. Offset Explorer

Install Offset Explorer, create a connection to localhost:9092, connect, and inspect topics, partitions, messages, and offsets. Exact fields vary by version.

H. NiFi

Class download: <https://archive.apache.org/dist/nifi/1.6.0/nifi-1.6.0-bin.zip>

1. Download and extract the ZIP.
2. Open the nifi-1.6.0 folder, then bin.
3. Run the startup script supplied in the distribution and wait for startup.
4. Open the class UI address: <http://localhost:9090/nifi/> or, if configured differently, <http://localhost:8098/nifi/>.
5. If neither works, check the startup logs for the actual UI address and port.

Note: NiFi 1.6.0 is an old release. These steps preserve the class setup; a new installation should use a supported release and its matching instructions.

I. Restarting Kafka

Class notes mention deleting a tmp folder before restarting. This is not a general Kafka requirement. Kafka stores data in configured log directories; deleting the wrong folder can remove topic data. Confirm the folder's purpose before deleting anything.

7. startingOffsets: earliest vs latest

In PySpark Structured Streaming, startingOffsets specifies where a **new streaming query** begins reading Kafka messages.

```
.option('startingOffsets', 'latest')
```

Mode	Starting position for a new query	Effect if 20 messages already exist
earliest	Earliest available offsets	Can read the 20 existing messages
latest	Latest offsets when query starts	Does not read messages from before its starting point

If five new messages arrive after the query starts, both modes can read those new messages, provided the query is running and the data is available.

Important: When a query restarts using its valid checkpoint, Spark resumes from checkpointed progress. The `startingOffsets` option does not override that progress.

8. checkpointLocation

A checkpoint is a persistent location where Spark Structured Streaming saves information about query progress and, when applicable, state needed for recovery.

```
option(
  'checkpointLocation',
  'file:///C:/tmp/kafka_topic1_checkpoint'
)
```

The checkpoint helps Spark track processed Kafka offsets, recover after failure, and maintain state for stateful operations. Reuse the same checkpoint to resume the same query.

If the checkpoint is deleted, Spark loses the previous query's saved progress. Starting again creates a new query from the position determined by `startingOffsets` and the available Kafka data. Do not delete a checkpoint if you intend to resume the same query.

Part 3 — Understanding the Class PySpark Code

Step 1: Create the Spark session

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark = SparkSession.builder.appName('window_functions') \
  .config(
    'spark.jars.packages',
    'org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.6'
  ) \
  .getOrCreate()
```

Creates or retrieves a Spark session and configures the Kafka connector. The connector version must be compatible with Spark and Scala.

Step 2: Read topic1 as a stream

```
df = spark.readStream.format('kafka') \
  .option('kafka.bootstrap.servers', 'localhost:9092') \
  .option('subscribe', 'topic1') \
  .option('startingOffsets', 'latest') \
  .load()
```

`readStream` creates a streaming DataFrame; `format('kafka')` selects Kafka; the bootstrap server and topic identify the source; `startingOffsets` sets the initial position for a new query.

Step 3: Cast key and value to strings

```
kafka_df = df.withColumn('key', col('key').cast('string')) \
  .withColumn('value', col('value').cast('string'))
```

Kafka's key and value columns are binary in the source schema. Casting them to strings makes text messages easier to process.

Step 4: Calculate length and uppercase the value

```
kafka_df1 = kafka_df.withColumn(
  'value_len', length(col('value'))
).withColumn(
  'value', upper(col('value'))
)
```

Adds `value_len` using the original message length, then converts value to uppercase. Example: hello kafka becomes HELLO KAFKA, with length 11.

Step 5: Write to the console

```
query = kafka_df1.writeStream.format('console') \
  .option(
    'checkpointLocation',
    'file:///C:/tmp/kafka_topic1_checkpoint'
  ) \
```

```
.outputMode('append') \
.start()

query.awaitTermination()
```

The console sink displays output. Append mode emits newly added rows. start() starts the query, and awaitTermination() keeps the application running.

Complete Flow

Kafka Producer → Kafka Topic/Broker → PySpark Structured Streaming → Console Output
 ■→ Checkpoint stores query progress

In this class code, the output goes to the console. A Delta Lake sink could be used in a separate configuration to persist processed records.

Quick Revision

Concept	Remember
Kafka	Streams and distributes event data
Delta Lake	Stores data in transactional tables
Producer	Sends messages
Consumer	Reads messages
Topic	Named stream of messages
Broker	Kafka server that stores and serves data
ZooKeeper	Coordination service for older Kafka setups
Pub/Sub	Publishers send to topics; subscribers consume
earliest	Earliest available offsets for a new query
latest	Latest offsets for a new query
checkpointLocation	Stores progress and recovery information
NiFi	Graphical dataflow automation
Offset Explorer	GUI for inspecting Kafka
PyCharm	IDE for Python/PySpark